ТЕХНИЧЕСКОЕ ЗАДАНИЕ

AI Coding Platform v2.1 — Local, Self-Hosted & GitHub Hybrid

Полный жизненный цикл AI-разработки (AI SDLC) | Проект: aitmaganbetov/ai-coding-platform

Статус: проектное ТЗ для реализации | Версия документа: 2.1 | Дата: 05.11.2026

1. Назначение и цели

Разработать самостоятельную платформу управления жизненным циклом программных проектов: от идеи и технического задания до разработки, аудита, модернизации, выпуска и сопровождения. Платформа должна работать локально без GitHub, на собственном сервере и в интеграции с GitHub Actions. Наличие LLM-провайдера не должно быть обязательным для запуска детерминированных проверок.

2. Границы проекта и режимы

Режим	Назначение	Основной результат
CREATE	Новый проект: требования, архитектура, план, каркас, код	Репозиторий и проверяемый MVP
DEVELOP	Новые функции и устранение дефектов	Отдельная ветка/PR с тестами
MODERNIZE	Рефакторинг, обновление зависимостей, миграция	Поэтапная модернизация с откатом
AUDIT	Технический, архитектурный, security и performance аудит	Отчёт с доказательствами и приоритетами
MAINTAIN	Плановое сопровождение и устранение регрессий	Контролируемые изменения и мониторинг

Вне v2.1: полностью автономный деплой в Production без согласования, гарантированное исправление всех уязвимостей, обучение собственной фундаментальной LLM, замена GitHub/GitLab как полноценного Git-хостинга.

3. Пользователи и права

Роль	Разрешения
Owner/Admin	Политики, проекты, провайдеры, секреты, исполнители, аудит
Maintainer	Управление задачами и подтверждение PR/релизов в пределах проекта
Developer	Создание задач, запуск анализов, просмотр результатов, предложение изменений
Auditor	Чтение отчётов, журналов и политик без права изменения
Agent	Ограниченные краткоживущие полномочия по конкретной задаче

RBAC применяется сервером, а не только интерфейсом. Межпроектная изоляция обязательна; каждая операция записывается с actor, project_id, run_id, timestamp, outcome.

4. Архитектура

- CLI: Python/Typer; REST API: FastAPI; Web UI: React (необязателен для CLI MVP).
- Core Orchestrator: конечный автомат задач, planner, router, policy engine, context manager, checkpoint manager, result aggregator.
- Executors: Local (ограниченный), Docker sandbox, GitHub Actions adapter; один интерфейс исполнения.
- LLM adapters: OpenAI, Anthropic, Ollama/vLLM; configurable models, token limits, fallback и отключаемый AI.
- Persistence: SQLite для single-user, PostgreSQL для multi-user; артефакты на локальном диске или MinIO; Redis опционально для очереди.
- Git adapters: локальный Git и GitHub API; интеграции должны быть заменяемыми.
- Security tools: Semgrep, Gitleaks, Trivy/OSV, linters, тестовые фреймворки и сборка; запускать реальные бинарные инструменты.

5. Оркестратор — обязательный модуль P0

Состояния: CREATED → DISCOVERY → PLANNED → AWAITING_APPROVAL → RUNNING → VERIFYING → COMPLETED; дополнительные FAILED, CANCELLED, PAUSED. Все переходы идемпотентны и журналируются. После рестарта выполнение продолжается с последнего подтверждённого checkpoint без повторного применения изменений.

- Task Planner строит DAG шагов с зависимостями, критериями успеха и бюджетом.
- Agent Router назначает агента по capability, доступу, стоимости и доступности.
- Context Manager индексирует репозиторий и передаёт минимальный релевантный контекст, исключая секреты.
- Executor ограничивает CPU/RAM, время, сеть, файловую систему и число повторов; команды allowlist/policy-based.
- Result Aggregator формирует единый итог и ссылки на доказательства; AI-вывод не может заменить статус реального теста.
- Human-in-the-loop: approval перед изменением критичных файлов, удалением, миграцией БД, merge и deployment.

6. AI-агенты и контракты

Агент	Вход	Выход
Requirements	Описание идеи, ограничения	ТЗ, user stories, acceptance criteria
Architect	ТЗ, текущее состояние	ADR, схема модулей, API/DB проект
Planner	ТЗ, ADR	Backlog, зависимости, оценка
Bootstrap	Утверждённый план	Каркас, CI, README, конфигурация
Developer	Issue, контекст	Patch/commit, пояснение, тесты
Reviewer	Diff, стандарты	Замечания с file/line/severity
Security	Diff, scanner reports	Уязвимости, evidence, рекомендации
QA	Требования, код	Тест-кейсы, результаты, coverage
Database	Schema, queries	Анализ миграций, индексов, транзакций
Performance	Метрики, профили	Bottlenecks и воспроизводимые бенчмарки

DevOps	Сборка, окружения	CI/CD, release plan, rollback
Maintenance	Issues, telemetry	План исправлений, проверяемые PR

Контракт каждого агента: task_id, input_refs, allowed_tools, scope, max_cost, max_duration, output_schema, evidence_refs, status, confidence. Выход валидируется JSON Schema; неверный ответ не считается выполненной задачей.

7. Функциональные требования (FR)

ID	Требование
FR-01	Импорт локальной папки/Git URL/GitHub repo; проверка доступа и состояния Git.
FR-02	Определение языка/стека: Laravel/PHP, React/TS, Python, Go, Rust; ручное переопределение.
FR-03	CREATE: диалог требований → версия ТЗ → approval → ADR → backlog → bootstrap.
FR-04	DEVELOP: Issue → план → ветка → patch → тесты → review → PR.
FR-05	MODERNIZE: baseline, impact analysis, малые PR, миграционные планы и rollback.
FR-06	AUDIT: read-only режим по умолчанию, находки с severity и evidence.
FR-07	MAINTAIN: triage дефектов, dependency updates, регрессионные тесты.
FR-08	Реестр агентов, capability routing, последовательные и параллельные шаги.
FR-09	Возобновление задач, retry с backoff, отмена и тайм-аут.
FR-10	Отчёты JSON/Markdown/SARIF/JUnit; комментарии к PR с защитой от дублей.
FR-11	Policy-as-code и блокирующие Quality Gates, независимые от AI.
FR-12	Локальная работа без GitHub и offline с локальной моделью при наличии.
FR-13	Self-Hosted многопроектный режим с RBAC, API и очередями.
FR-14	Подключение GitHub Actions reusable workflows через workflow_call.
FR-15	Настройка бюджета токенов/стоимости, лимитов и маршрутизации моделей.
FR-16	Протоколирование решений, команд, версий инструментов и артефактов.
FR-17	Управляемое создание PR; запрет auto-merge и production без approval.
FR-18	Маскировка секретов и фильтрация контекста перед вызовом внешних LLM.

8. Нефункциональные требования (NFR)

ID	Критерий

NFR-01	CLI поддерживает Linux, Windows, macOS; контейнерный сервер — Linux x86_64.
NFR-02	Не менее 10 параллельных заданий на сервере при выделенных ресурсах; лимит конфигурируемый.
NFR-03	Сбой исполнителя не теряет подтверждённые состояния и артефакты.
NFR-04	Для API и UI: аутентификация, RBAC, TLS, rate limiting, audit trail.
NFR-05	Результаты проверок воспроизводимы на закреплённых версиях инструментов.
NFR-06	Модульные тесты core ≥80% покрытия; критические state transitions — 100% ветвей по тест-кейсам.
NFR-07	Логи без секретов, структурированные метрики и trace/run_id.
NFR-08	Пользователь может экспортировать данные проекта и удалить его по политике хранения.

9. GitHub Actions

Reusable workflows: quality, security, test-php, test-node, test-python, test-go, test-rust, ai-review, release. Caller workflows используют workflow_call и минимальные permissions. Версии сторонних Actions закреплять на SHA; при доступе к секретам использовать environments и approval. Для fork PR — непривилегированный read-only workflow; не запускать недоверенный код с pull_request_target и write-token. Self-hosted runners использовать только с доверенными заданиями и изоляцией.

10. Quality Gates и правила принятия

Gate	Блокирует merge при
Build	Ошибка сборки или установки зависимостей
Tests	Падение обязательных тестов
Static	Ошибки линтера/типизации выше заданного порога
Security	Подтверждённые Critical/High по политике и новые секреты
Coverage	Снижение покрытия критичных модулей ниже порога
Review	Отсутствие требуемого человеческого approval

AI Review носит рекомендательный характер; блокировка возможна только при явной утверждённой политике и проверяемых результатах. Baseline существующих проблем допускается с исключениями, владельцем и сроком действия.

11. Конфигурация проекта

Файл .ai-coding.yml: schema_version, project, mode, stacks, execution.backend, ai.provider/model, agents.enabled, security.network, quality_gates, budgets, approvals, artifacts.retention. Конфигурация проходит schema validation. Секреты передаются только через secret store / environment, никогда в YAML.

12. Безопасность и модель угроз

- Недоверенные README, issues, diff, зависимости и тестовые данные рассматриваются как возможные prompt injection.
- Права агентам выдаются по принципу least privilege; shell/network/file write запрещены по умолчанию до назначения роли.
- Исполнение кода — в изолированных контейнерах без privileged, без Docker socket, с read-only mounts там, где возможно.
- Внешние LLM используются только после проверки политики отправки кода; режим local-only полностью запрещает egress.
- Секреты не включать в логи, prompts, PR-комментарии и артефакты; реализовать redaction и секрет-сканирование.
- Подписанные/проверенные артефакты, SBOM, provenance, контроль цепочки поставок и журнал изменений.
- Удаление/миграции/деплой — только по approval; rollback и backup обязательны для рискованных операций.

13. API и модель данных

Основные сущности: users, projects, repositories, runs, tasks, task_dependencies, agents, agent_executions, approvals, artifacts, findings, policies, model_usage, audit_events, checkpoints. Идентификаторы UUID, время UTC, optimistic concurrency/version для состояния задач.

REST API: POST /projects; POST /projects/{id}/runs; GET /runs/{id}; POST /runs/{id}/approve; POST /runs/{id}/cancel; GET /runs/{id}/artifacts; GET /projects/{id}/findings; GET /audit-events. Операции создания идемпотентны по Idempotency-Key.

14. CLI интерфейс

Команды: ai-coding init, project add, create, develop, modernize, audit, maintain, run status, run resume, run cancel, config validate, agents list, report export. Для изменяющих операций поддерживать --dry-run и --require-approval.

15. Развёртывание

Профиль	Компоненты
Local Lite	CLI + SQLite + Git worktree + локальный sandbox; LLM опционально
Self-Hosted	FastAPI + worker + PostgreSQL + artifact store + Web UI + Docker executor
GitHub Hybrid	Self-Hosted или CLI + GitHub App/token + reusable workflows + GitHub Rulesets

GitHub Actions не является автономным offline-оркестратором; offline режим должен выполняться через локальный scheduler/executor. В многопользовательском режиме обязательны изоляция проектов, резервное копирование БД и контролируемая ротация ключей.

16. Этапы разработки и зависимости

Спринт	Содержание	Результат/приёмка
S1 — 2 нед.	Архитектура, ADR, схемы, CLI, config schema, Git abstraction	init/config validate; тесты каркаса
S2 — 2 нед.	Orchestrator FSM, persistence, checkpoint, retry, policy	Восстановление после прерывания; unit tests

S3 — 2 нед.	Local/Docker executor, sandbox, tool registry	Безопасный локальный audit
S4 — 2 нед.	GitHub adapter, reusable workflows, PR/report integration	CI проходит на пилотном репозитории
S5 — 2 нед.	CREATE: requirements, architect, planning, bootstrap	Новый проект из задачи до PR
S6 — 2 нед.	DEVELOP и MODERNIZE, AI review/fix, approvals	Функция/рефакторинг через отдельный PR
S7 — 2 нед.	AUDIT/MAINTAIN, security tools, reports, QA	Воспроизводимые результаты сканеров
S8 — 2 нед.	Self-Hosted API, RBAC, Web UI, observability	Два проекта и несколько пользователей
S9 — 2 нед.	Hardening, load/security tests, docs, release	Приёмочные сценарии и v2.1

Оценка 18 недель для небольшой команды — плановый ориентир, не гарантия; возможен выпуск промежуточного Local MVP после S3–S4.

17. Приёмочные сценарии

- AC-01: Создать новый проект из текстового описания; получить согласованное ТЗ, ADR, backlog, bootstrap и PR.
- AC-02: Подключить существующий Laravel+React проект; выполнить read-only audit и получить подтверждённые находки с путями файлов.
- AC-03: По Issue добавить функцию в отдельной ветке, выполнить тесты, сформировать PR без прямого push в main.
- AC-04: Смоделировать падение worker и убедиться, что run возобновляется без дублирования коммитов.
- AC-05: Подключить локальную модель и выполнить audit без внешних сетевых запросов.
- AC-06: Уязвимый PR не проходит Quality Gate; человек видит причины и доказательства.
- AC-07: Попытка prompt injection из README не даёт агенту дополнительных прав.
- AC-08: Запуск на self-hosted сервере с двумя проектами подтверждает RBAC и отсутствие утечки контекста.
- AC-09: Подключить GitHub Actions к двум разным стекам, получить статусы проверок в PR.
- AC-10: Проверить dry-run, ручное approval, отмену и экспорт audit trail.

18. Риски и меры

Риск	Митигирующая мера
Prompt injection	Недоверенный контент, tool policy, изоляция, approvals
Утечка исходного кода	Local-only policy, redaction, provider allowlist
Повреждение репозитория	Git worktree, отдельная ветка, diff review, rollback
Нестабильность LLM	Schema validation, deterministic gates, retry budget
Высокие расходы	Token caps, caching, routing, usage reports
Долгие CI задания	Параллелизм, incremental scans, timeouts
Уязвимые runners	Эфемерные runner, разделение trust zones

19. Приоритетный backlog P0

- P0-01: Создать репозиторий, CI и кодстайл.
- P0-02: CLI и конфигурационная схема.
- P0-03: State machine + SQLite persistence.
- P0-04: Sandbox executor и Git worktree.
- P0-05: Policy/approval engine.
- P0-06: GitHub reusable workflows + checks.
- P0-07: Agent contracts и provider adapters.
- P0-08: Audit tools + отчёты JSON/Markdown.
- P0-09: DEVELOP end-to-end через PR.
- P0-10: CREATE минимальный end-to-end.

20. Определение готовности (Definition of Done)

Функция считается завершённой только при наличии кода, тестов, обновлённой документации, безопасных значений по умолчанию, прохождения CI, проверенного сценария отказа (если применимо), отчёта и одобрения ответственного. Выпуск v2.1 допускается после прохождения всех AC-01…AC-10, закрытия блокирующих дефектов и подготовки эксплуатационной инструкции.

Конец документа. Изменения требований оформлять через ADR/Change Request с оценкой влияния на сроки и безопасность.